

IP Packets & Network Protocols Notes

IP/Data Packets

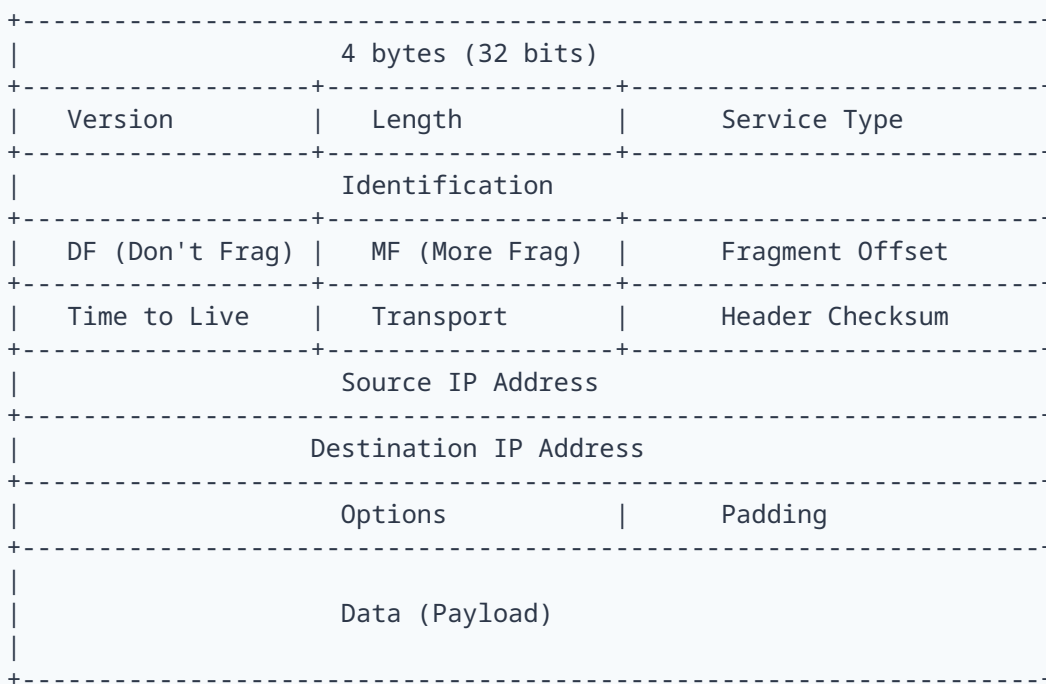
What is an IP Packet?

A packet is the smallest unit of data sent over a network. In most computer networks, data is broken down into small units called packets and sent between devices. The same holds true on the internet.

Each packet has a specific structure, defined by a protocol, that allows the recipient to understand the packet's contents. An IP packet is a unit of data in a network that contains information about the source and destination addresses and other control information needed to transport the packet over a network.

IP Packet Structure Diagram

Below is the structural layout of an IP Packet Header (spanning 24 bytes total):



Packet Encapsulation & Reassembly

Data is broken into packets, transmitted, then reassembled. Every packet has two main components:

- **[Headers] + [Payload]**
- **Headers:** Contains control info.
- **Payload:** Contains actual data.

Each packet contains both data and control information. The data portion (payload) contains the actual content being transmitted, such as text, images, or video. While the control information in the packet header

includes details such as the source and destination IP addresses, time to live, packet length, and routing instructions.

How Data Moves Through Layers:

Each layer adds its own header:

[Ethernet] [IP] [TCP] [Data]

This process is called **encapsulation**.

Structure of an IP Packet Header

Several fields in an IP packet header help route and deliver the packet to its desired destination:

- **Version:** Indicates the IP protocol version, such as IPv4 or IPv6, which helps determine how the packet should be routed.
- **Total Length:** Specifies the packet's total length of the data payload (the message) and the header, which are essential for the recipient to understand the packet's contents.
- **Protocol:** Determines the high-level protocol, like TCP, UDP, etc., used in the data payload.
- **Time to Live (TTL):** Used to prevent packets from circulating indefinitely, which could cause network congestion. Every time a packet gets forwarded, the router decreases the TTL by one, and when the TTL becomes zero, the packet gets discarded, helping to avoid congestion.
- **Source IP Address:** Specifies the IP address of the device that sent the packet. It's used to reply to the sender, and if a router cannot send the packet to the following location, it sends a message to the source using the source IP address to indicate the problem.
- **Destination IP Address:** Identifies the IP address of the intended recipient of the packet. Routers use it to determine where to forward the packet to reach its final destination.

IPv4 Header Fields Reference (Layer 3 Network Layer)

Field	Meaning	SOC Use
Version	IPv4 or IPv6	Protocol identification
Source IP	Sender	Identify attacker
Destination IP	Receiver	Target system
TTL (Time To Live)	Packet lifetime	Detect anomalies
Protocol	TCP / UDP / ICMP	Traffic type
Total Length	Packet size	Detect exfiltration
Header Checksum	Integrity	Corruption detection

TCP Header Reference (Layer 4)

Field	Meaning	SOC Use
Source Port	Sender port	Identify origin
Destination Port	Target service	Detect attack
Sequence Number	Order tracking	Session analysis
ACK Number	Confirmation	Flow validation
Flags	Connection state	Attack detection
Window Size	Flow control	Traffic behavior

Packet Analysis Example

Example Packet Specifications:

- Source IP: 192.168.1.10
- Destination IP: 45.77.123.66
- Protocol: TCP
- TTL: 64
- Length: 1500 bytes

SOC Analysis:

- **Source IP:** Internal → normal. External → suspicious.
- **Destination IP:** Known service → OK. Unknown / malicious → alert.

TTL - Time To Live

Time to live (TTL) refers to the amount of time or "hops" that a packet is set to exist inside a network before being discarded by a router. TTL is also used in other contexts including CDN caching and DNS caching.

TTL Mechanism Image Representation

```
Client
|
+----> [ Router 1 ] (TTL = 4)
      |
      +----> [ Router 2 ] (TTL = 3)
            |
            +----> [ Router 3 ] (TTL = 2)
                  |
                  +----> [ Router 4 ] (TTL = 1) ---> [ Destination Host ]
```

(Image Source: cloudflare.com)

How TTL Works:

1. When a packet of information is created and sent out across the Internet, there is a risk that it will continue to pass from router to router indefinitely.
2. To mitigate this possibility, packets are designed with an expiration called a time-to-live or hop limit. Packet TTL can also be useful in determining how long a packet has been in circulation, and allow the sender to receive information about a packet's path through the Internet.
3. Each packet has a place where it stores a numerical value determining how much longer it should continue to move through the network.
4. Every time a router receives a packet, it subtracts one from the TTL count and then passes it onto the next location in the network.
5. If at any point the TTL count is equal to zero after the subtraction, the router will discard the packet and send an ICMP message back to the originating host.

Diagnostic Tools and TTL:

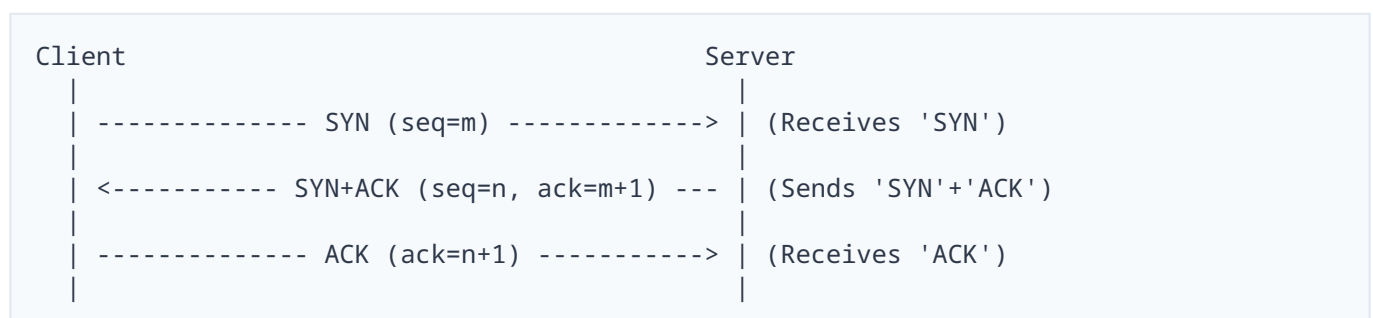
The commonly used network commands **ping** and **tracert** both utilize TTL. When using the **tracert** command, a stream of packets with increasingly higher sequential TTLs are sent across the Internet towards a destination.

Because each step along the connection is the last stop for one of the packets, each location will return an ICMP message to the sender after discarding the packet. The time it takes for the ICMP message to return to the sender is then used to determine how long it takes to get to each successive hop along the network.

TCP 3-Way Handshake

The handshake is a data-sharing process that involves a back-and-forth communication between a client (for example a mobile phone or Smart TV) and a server. The purpose of this communication is to establish a secure and reliable connection so that data (files, images, videos, etc.) can be transferred efficiently between the systems.

3-Way Handshake Image Sequence



- **Step 1:** Client sends **SYN** (seq=m).
- **Step 2:** Server receives **SYN****, sends ****SYN+ACK**** (seq=n, ack=m+1).
- **Step 3:** Client receives **SYN+ACK****, sends ****ACK**** (ack=n+1). Connection is established.

TCP Flags Reference & SOC Analysis

Flag	Meaning	SOC Use
SYN	Start	Connection attempts
ACK	Confirm	Normal traffic
FIN	Close	Session end
RST	Reset	Abnormal termination
PSH	Push data	Immediate delivery
URG	Urgent	Rare

Suspicious Flag Patterns For SOC:

Pattern	Meaning
SYN only	SYN flood attack
SYN + FIN	Malicious / Scanning activity
RST flood	Connection disruption
No ACK	Half-open connection attack

Real SOC Scenario

Observed Event Sequence:

- Source IP: 185.221.x.x
- Packets Received: 100,000 SYN packets
- ACK Count: 0
- Destination: Port 80

Conclusion: SYN Flood DDoS Attack.